

COMPX₃₄₁ - Advanced Software Engineering Design



THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato

Software Development Models

COMPX₃₄₁

Overview

- Engineering development models
- Software Engineering development models
- Waterfall
- Agile

Engineering development models

- A development model is:
 - A way of organising the work of individuals or teams on a project
 - It can involve documents, processes, techniques, and standards
- The use of development models in Software Engineering stem from the use of development models in other fields such as Engineering
- In Engineering, this involves applying science and mathematics to:
 - Find novel solutions
 - Improve existing solutions

Software engineering development models

- Software Engineering lacks the clear and generally agreed processes that other engineering disciplines have developed over long periods of time
- As a result, there is no one software engineering development model. Instead numerous models have been developed over time.

Software development models

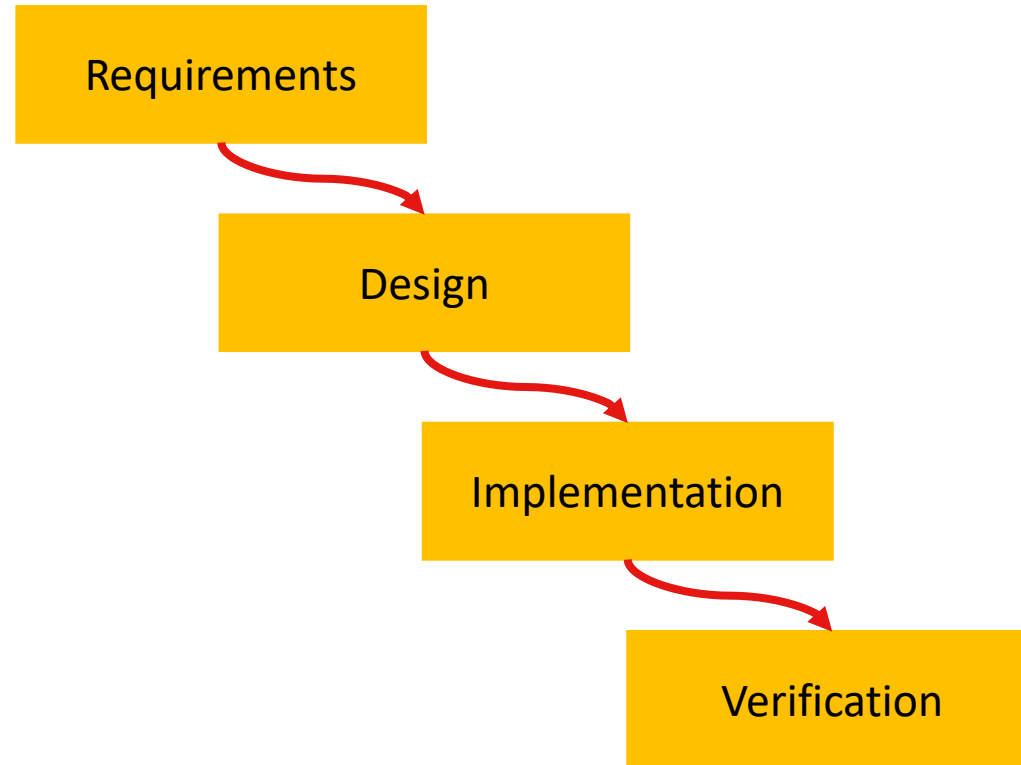
1. Formal specifications / proofs (From Mathematics)
 - Automated checking of programs
2. Rigorous documentation and planning (From other engineering disciplines)
 - Waterfall process
3. Processes (From management and process control)
 - Testing / code review / programming languages / coding standards (DevOps ish)
4. Agile methods (Sort of biological ???)
 - Iterative and incremental development / SCRUM / extreme programming

Waterfall

Waterfall

- Waterfall Software Engineering development model
 - A linear, sequential approach
 - Each phase depends on the deliverables of the previous one

Waterfall



Waterfall

- Originally used in Engineering disciplines
 - Manufacturing and construction industries
- For Software:
 - 1956 – the first waterfall-ish approach was presented (SAGE)
 - 1970 – the first formal description of the waterfall model (Royce)
 - 1976 – the first time the term “waterfall” was used
 - 1985 - the United States Department of Defence adopts waterfall as their standard for working with software development contractors

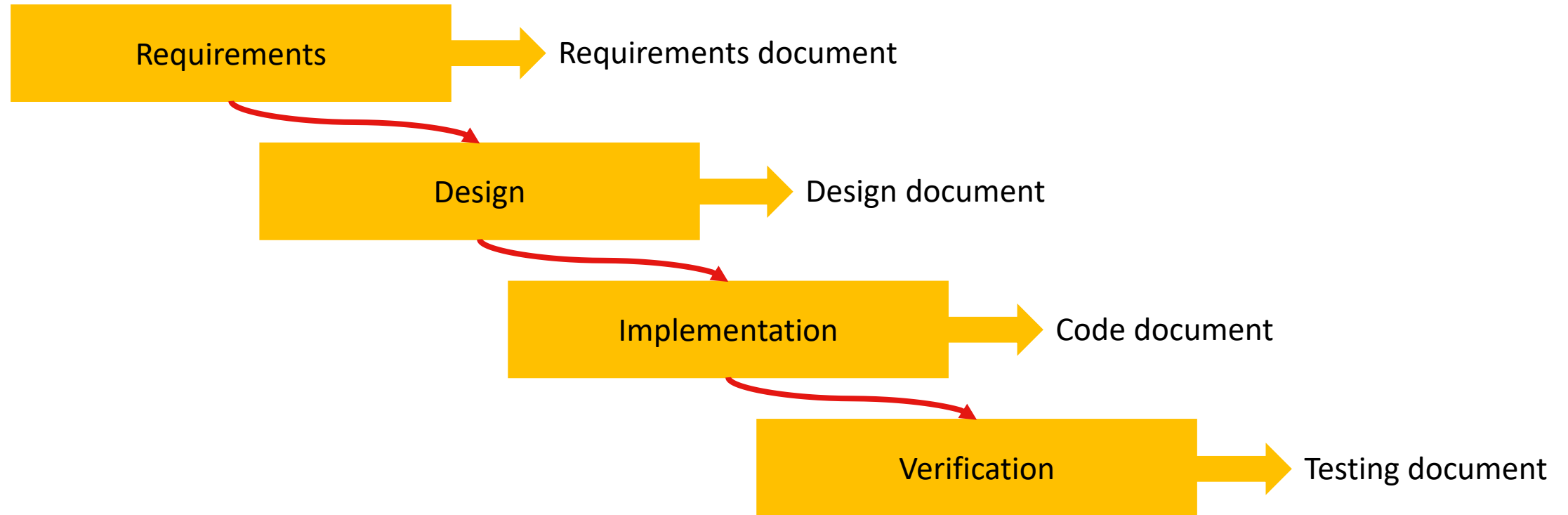
Waterfall

- Fun fact: the 1970 paper that provided the first formal description of the waterfall model was actually using it as an example of a flawed, non-working model.
- Royce 1970 <http://www-scf.usc.edu/~csci201/lectures/Lecture11/royce1970.pdf>

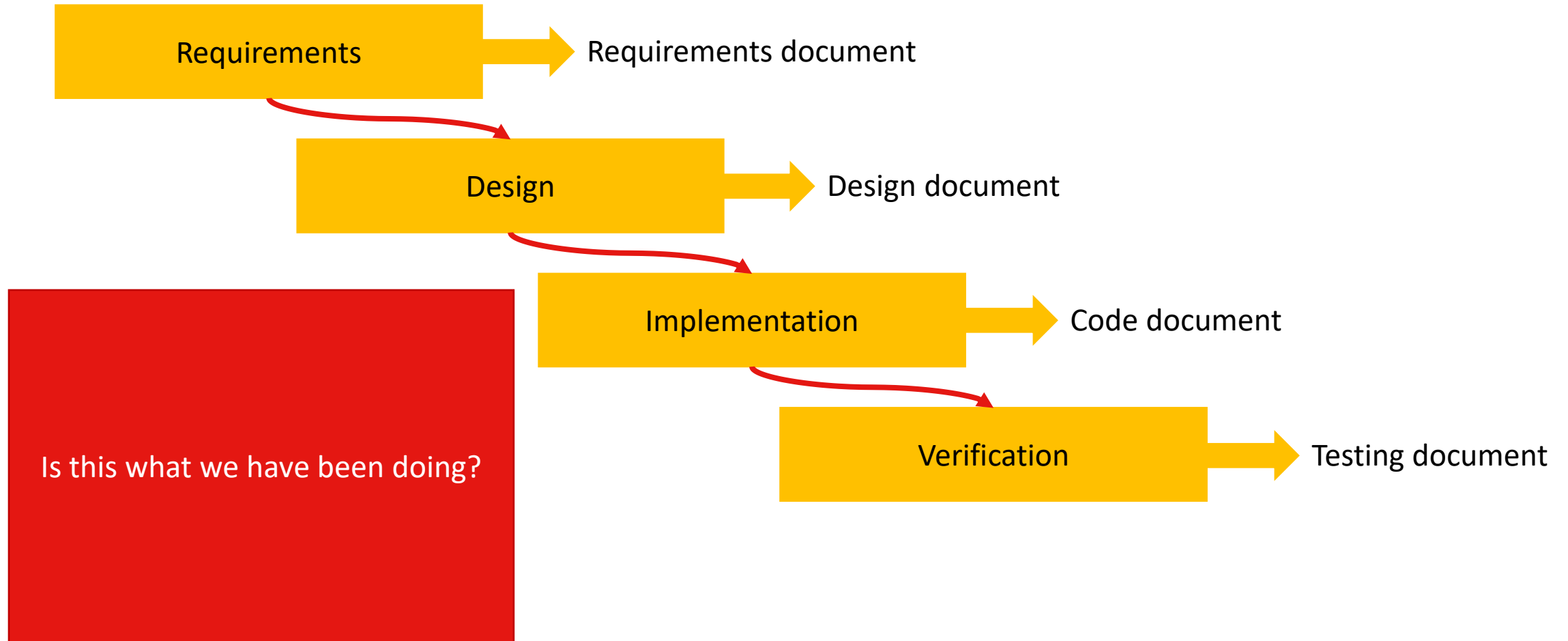
Waterfall

1. Requirements
 - Functional (the features that the application should include)
 - Non functional (hardware specifications, security, etc.)
2. Design
 - Data structures, storage, classes
3. Implementation
 - Coding
4. Verification
 - Testing

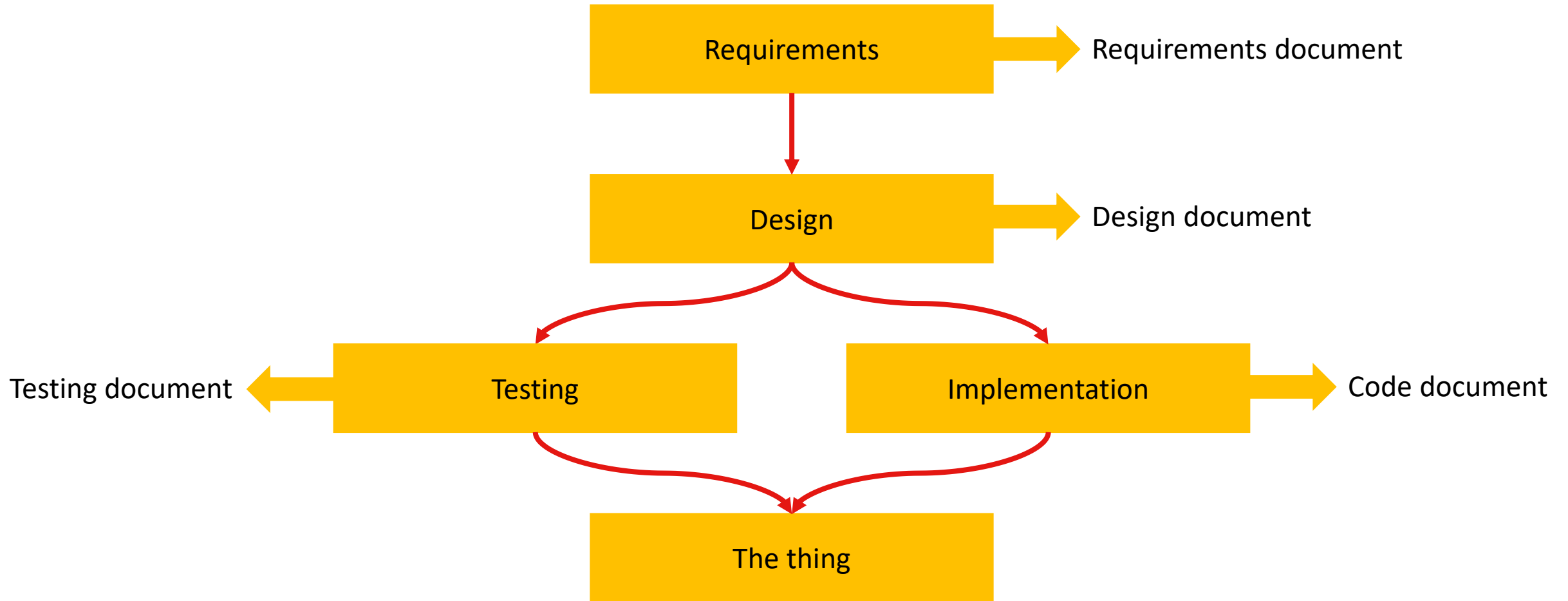
Waterfall



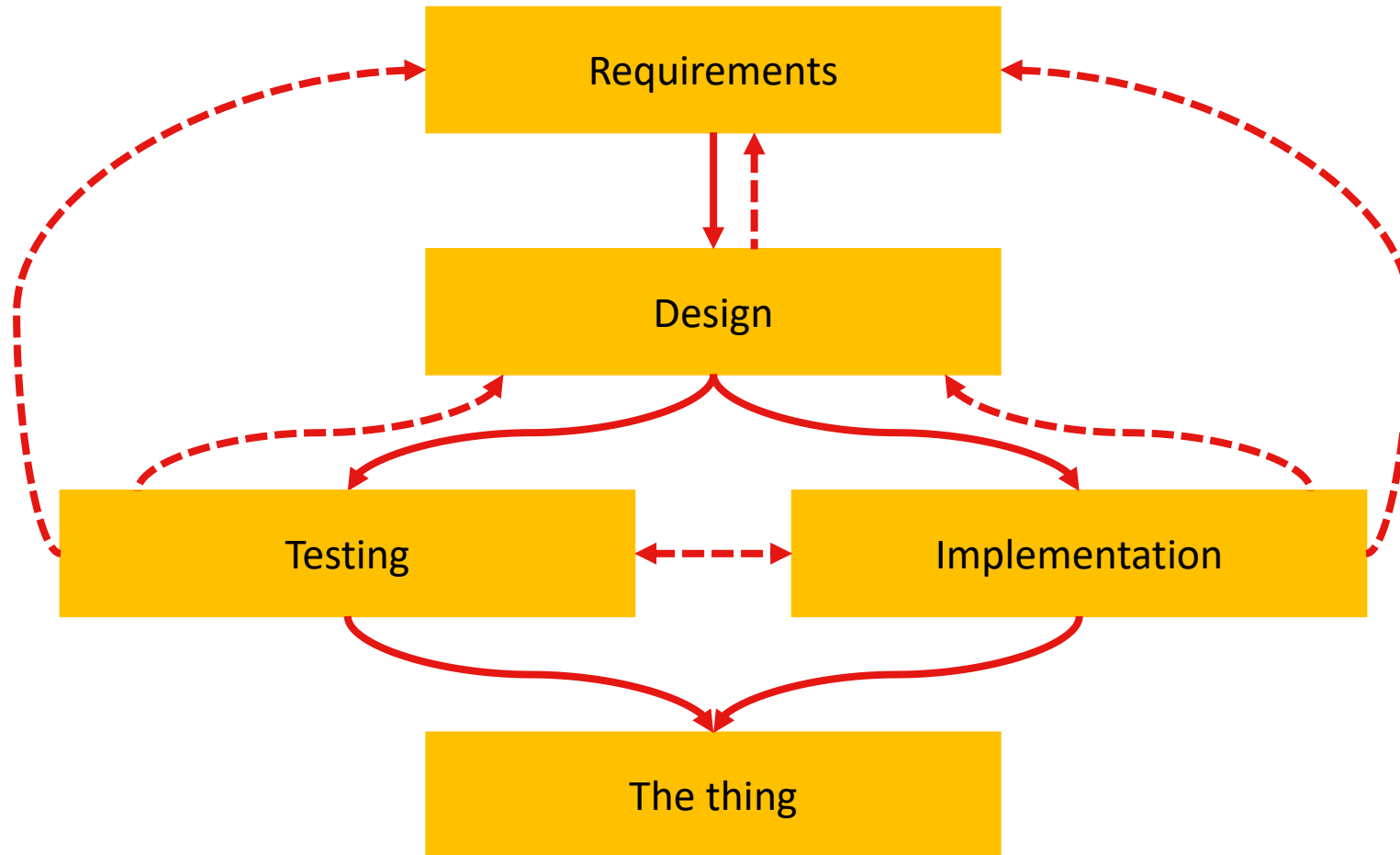
Waterfall



(?)Waterfall



(Not) Waterfall

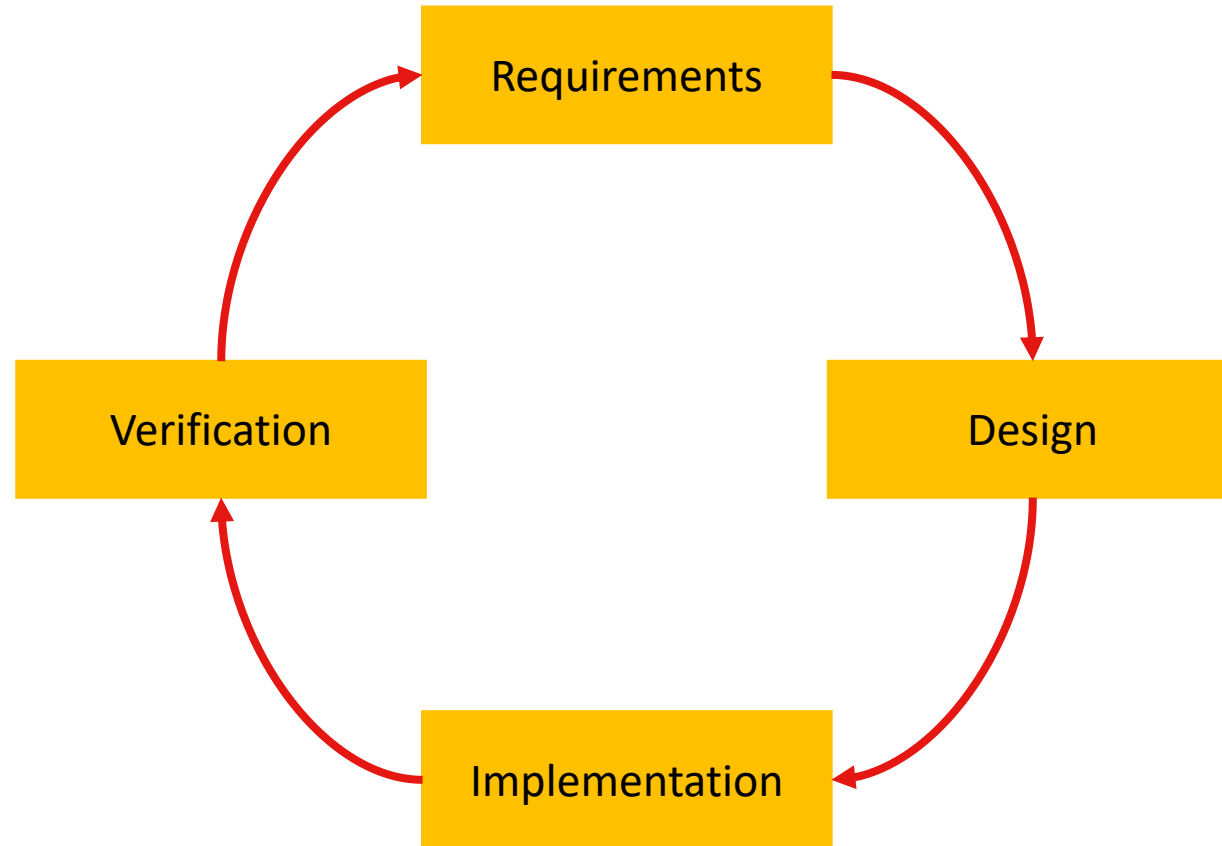


Agile

Agile

- Agile Software Engineering development models
 - Agile in general refers to development models that evolve software rather than require complete upfront design
 - Iterative and incremental

Agile



Agile

- The key observation is that we often can't fully understand a project and its requirements at the start and that we need methods that cope with change

Agile

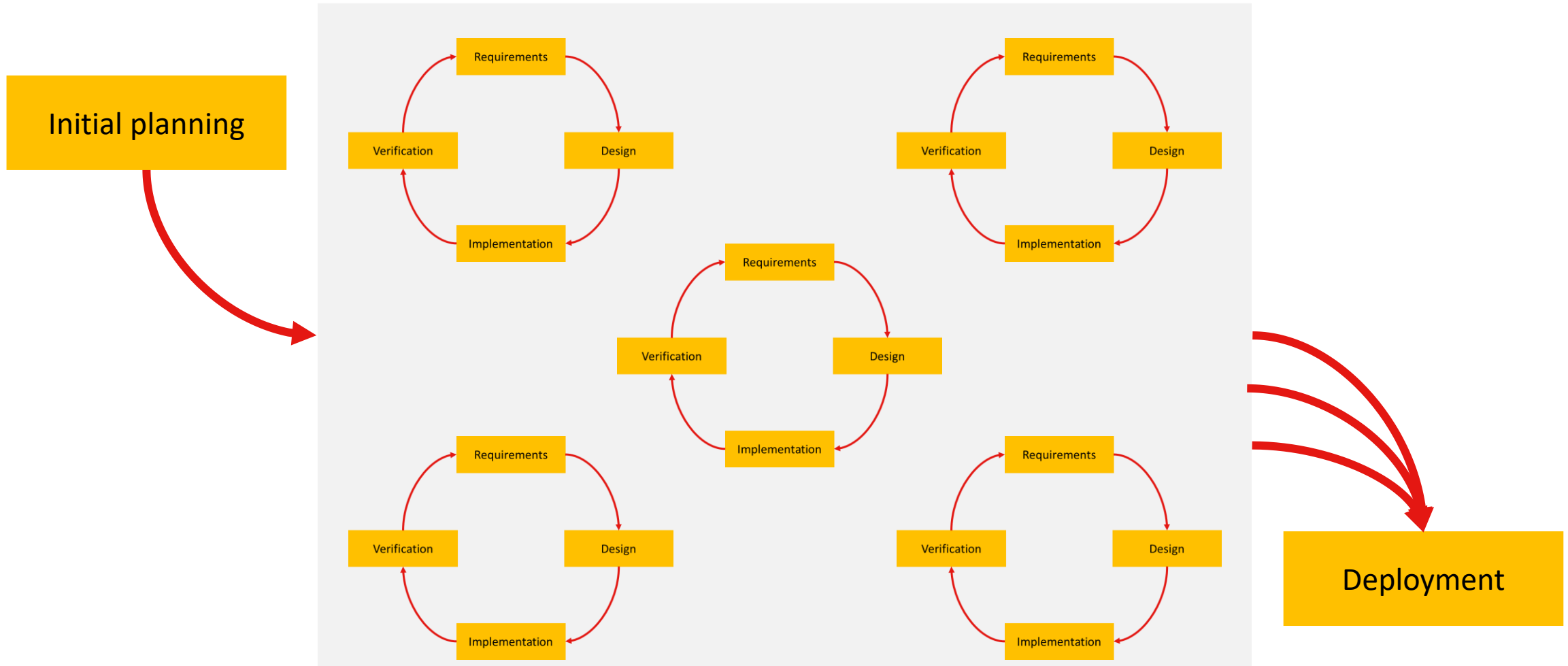
Agile has been around for a long time.

- 1957 – Iterative and incremental development methods (IID) (1 year after WF)
- 1960s – NASA used agile (IID) methods for Project Mercury
- 1970s – adaptive software development
- 1977–1980 – NASA's space shuttle software
- 1990s – a surge in agile methods came about as a “backlash” to waterfall
- 1994 – the United States Department of Defence adopts an agile methodology as their standard for working with software development contractors

Agile

- Breaking a complex task down into smaller, more manageable chunks.
 - Multiple iterations: requirements, design, implementation, and testing
 - Allows for changes throughout the process
 - Smaller chunks can be understood separately

Agile



Agile example

- You have been asked to develop a VERY basic social media platform.

Agile example

1. Initial planning

- Create a preliminary release plan
 - Must haves
 - Register/login/logout, create profile, post text, post images
 - Should haves
 - Post videos, comment on posts
 - Could haves
 - Likes, follows

Agile example

2. Iteration A

- Focused on “registering”
 - Requirements
 - Design
 - Implementation
 - Testing

Agile example

3. Iteration B

- Focused on “login/logout”
 - Requirements
 - Design
 - Implementation
 - Testing

Agile example

4. Iteration C

- Focused on “creating a profile”
 - Requirements
 - Design
 - Implementation
 - Testing

Agile example

- Iterations can be completed simultaneously by different groups of developers
 - Frequent integration
 - Frequent testing
- Finished features can be deployed (released) while other features are still under development.
- If you find a mistake, you do not have to start from the beginning again

Agile example

- This is a “generalized” description of Agile
- There are A LOT of different Agile development models
 - Iterative and incremental
 - Extreme programming
 - SCRUM
 - ...

Iterative and Incremental

Iterative and Incremental

- Develop a system through
 - Repeated cycles (iterative)
 - And in smaller portions (incremental)

Iterative and Incremental

- Three stages:
 - Initialization step
 - Project control list (increments)
 - Iteration step

1. Initialization

The initialization step creates a base version of the system.

- Provides a product to which the user can react
- Offers some of the key aspects
- Must be simple enough to understand and implement easily

2. Project Control List

The project control list breaks the project into increments (tasks)

- Contains a record of all tasks to be performed
- Includes new features
- And redesigns of existing features
- Constantly revised

3. Iteration

- Involves iterating through phases to complete tasks from the project control list
- Each phase can be iterated over 1 or more times (time boxed), revising the project control list as needed

3. Iteration

- Four phases:
 - Inception (scope, requirements, risks)
 - Elaboration (architecture)
 - Construction (production ready code)
 - Transition (release)

3. Iteration

- Four phases:
 - Inception (scope, requirements, risks) (Requirements)
 - Elaboration (architecture) (Design)
 - Construction (production ready code) (Implementation)
 - Transition (release) (Testing and release)

Extreme Programming (XP)

Extreme Programming

- Advocates frequent "releases" in short development cycles
 - reduce the cost of changes in requirements
 - introduce checkpoints where new requirements can be added
 - improve productivity
- Does not define a stable set of requirements
- Changes are natural, inescapable and desirable

Extreme Programming

- Elements of Extreme Programming (XP)
 - extensive code review
 - programming in pairs (peer programming)
 - unit testing of all code
 - avoiding programming of features until they are actually needed
 - expecting changes in the customer's requirements as time passes and the problem is better understood
 - frequent communication with the customer and among programmers

Extreme Programming

- There are four “basic activities” performed when using XP:
 1. Coding
 2. Testing
 3. Listening
 4. Designing

1. Coding

XP doctrine: The only truly important product of the system development process is
CODE

- Use code to
 - Figure out the most suitable solution
 - Communicate thoughts about programming problems
 - Code is always clear and concise and cannot be interpreted in more than one way
 - Other programmers can give feedback by also “coding their thoughts”

1. Coding

- Coding Rules
 - The customer is always available
 - Code the unit test first
 - Only one pair integrates code at a time
 - Leave optimization until last
 - No overtime

2. Testing

- XP's approach is that if a little testing can eliminate a few flaws, a lot of testing can eliminate many more flaws.
 - Unit tests
 - Acceptance tests
 - Integration testing

2. Testing

- Testing Rules:
 - All code must have unit tests
 - All code must pass all unit tests before it can be released.
 - When a bug is found, tests are created before the bug is addressed (a bug is not an error in logic; it is a test that was not written)
 - Acceptance tests are run often and the results are published

3. Listening

Programmers must listen to what the customer needs the system to do and give the customer feedback

- The planning game
 - A meeting that occurs ~ once a week
 - Release planning (customer and developer)
 - Iteration planning (developer)

4. Designing

- XP is centred on coding, testing, and listening
 - Argued that if it is done well, design is not needed
 - Sufficient for smaller projects ...
- BUT
 - in reality, a system becomes too complex, or dependencies become unclear

4. Designing

- Designing involves:
 - Creating a design structure
 - Avoiding lots of dependencies
 - Ensuring that changing one part of a system won't affect other parts

XP Values

- Communication
 - Between customers and programmers
 - As opposed to documentation
- Simplicity
 - Starting with the simplest solution
 - Developing and coding for the needs of today
- Feedback
 - From the system (unit tests)
 - From the customer (acceptance tests)
 - From the team (planning game)
- Courage
 - Knowing when to throw code away
- Respect
 - Your own work (strive for high quality)
 - The work of others (don't commit changes that break existing code)

SCRUM

Scrum

- Scrum is a lightweight, iterative and incremental framework for managing complex work.

Scrum

- Key principles:
 - Customers will change their minds
 - There will be unpredictable changes
 - Accepts that a problem cannot be fully understood or defined up front
 - Quick deliverables
 - Responding to emerging requirements
 - Adapts to evolving technologies

Scrum

- Key Features:
 - Roles
 - Workflow (Sprints)
 - Artefacts (Backlog, Burn down charts, etc.)

Scrum roles

- Product owner
 - Voice of the customer
- Development team
 - 3-9 members
- Scrum master
 - Buffer between the team and distracting influences

Scrum workflow

- Sprint
 - Time-boxed development
 - Between 1 week and 1 month

Sprint meetings

- Sprint planning
 - ~ 4 hours (for a 2 week sprint), at the start of a sprint
 - Select items from the product backlog
 - Define sprint backlog
- Daily scrum
 - Same time and place every day
 - What did I complete yesterday?
 - What do I plan to complete today?
 - Any impediments?

Sprint meetings

- Sprint review
 - ~ 2 hours (for a two week sprint), at the end of a sprint
 - Review completed work
 - Present work to stakeholders and decide what's next
 - Incomplete work is NOT presented
- Sprint retrospective
 - ~ 1.5 hrs (for a two week sprint), at the end of the sprint
 - What went well during the sprint?
 - What did not go well?
 - What could be improved?

Scrum artifacts

- Product backlog
- Sprint backlog
- Sprint burn-down chart
- Release burn-up chart

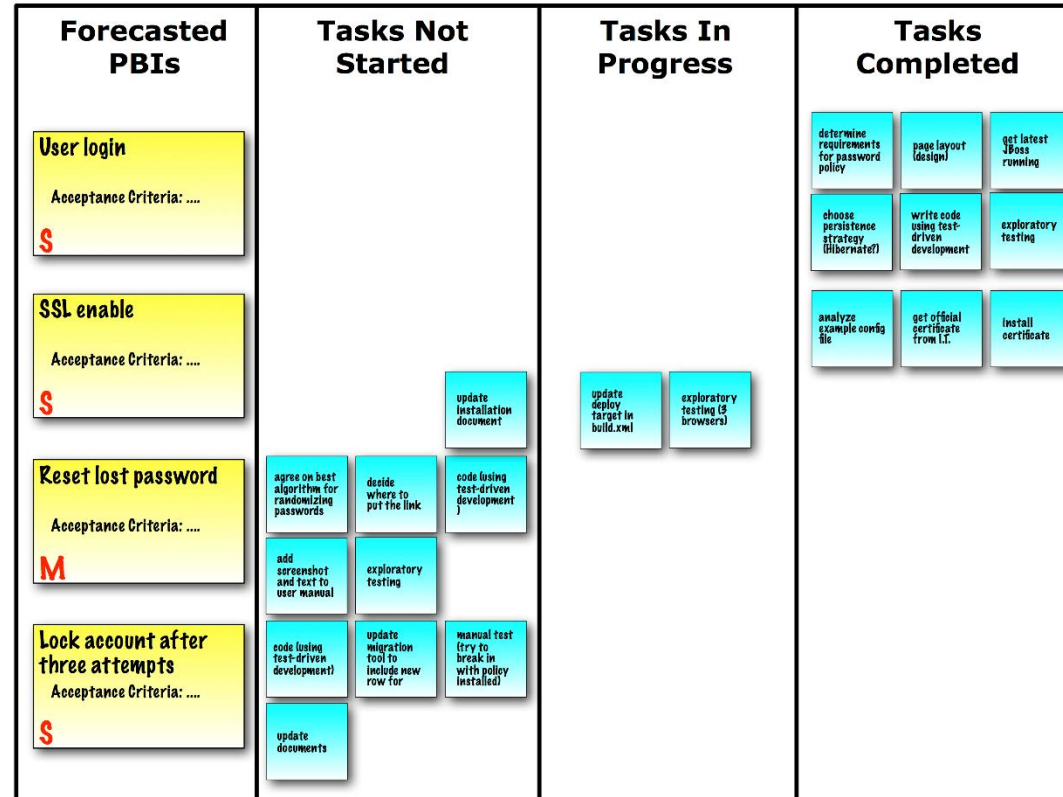
Product backlog

- Product backlog
 - A breakdown of the work that needs to be done
 - Product requirements
 - I.e. user stories, use cases
 - Define features, bug fixes, non-functional requirements
 - Controlled by the “product owner”

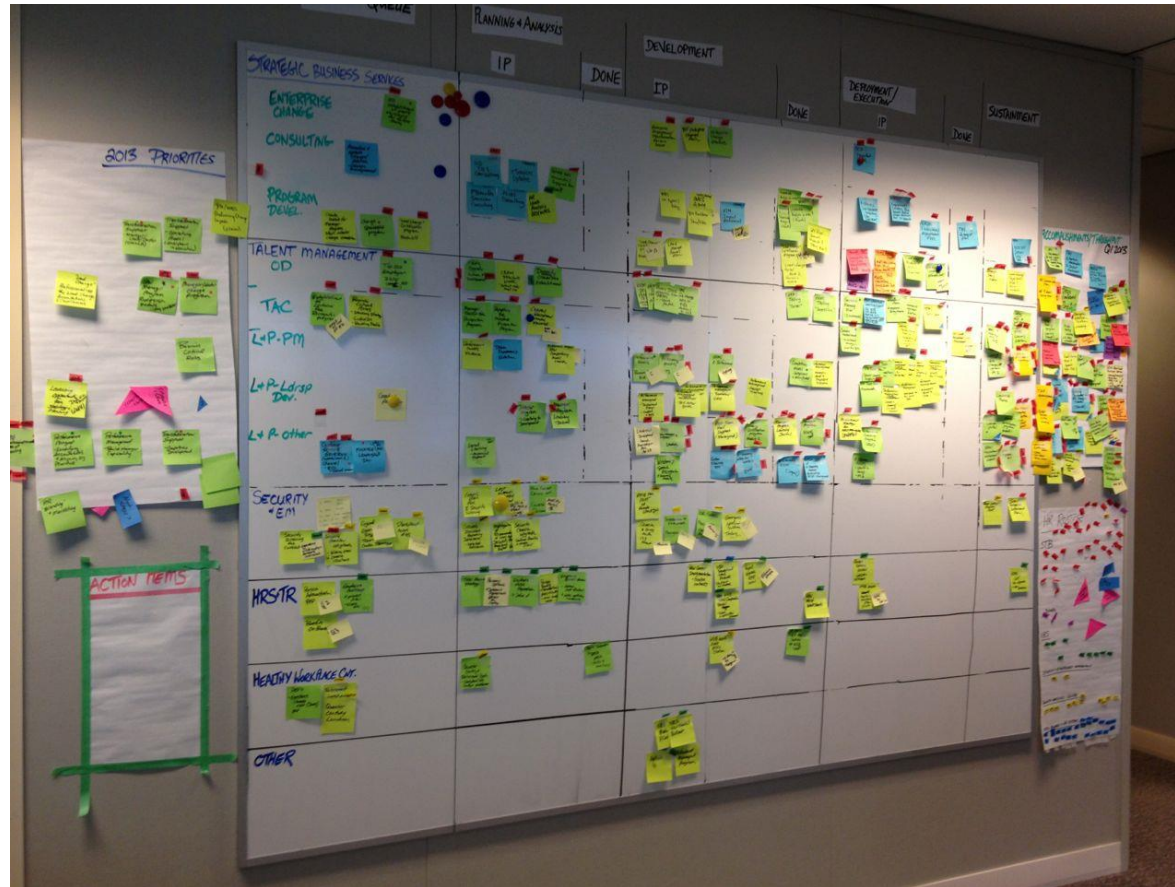
Sprint backlog

- Sprint backlog
 - The work to be addressed during the next sprint
 - Derived from items in the product backlog
 - Broken down into tasks
 - Team members “sign up” to complete a task
 - The full sprint backlog is not always completed in one sprint

Sprint backlog (ideal)



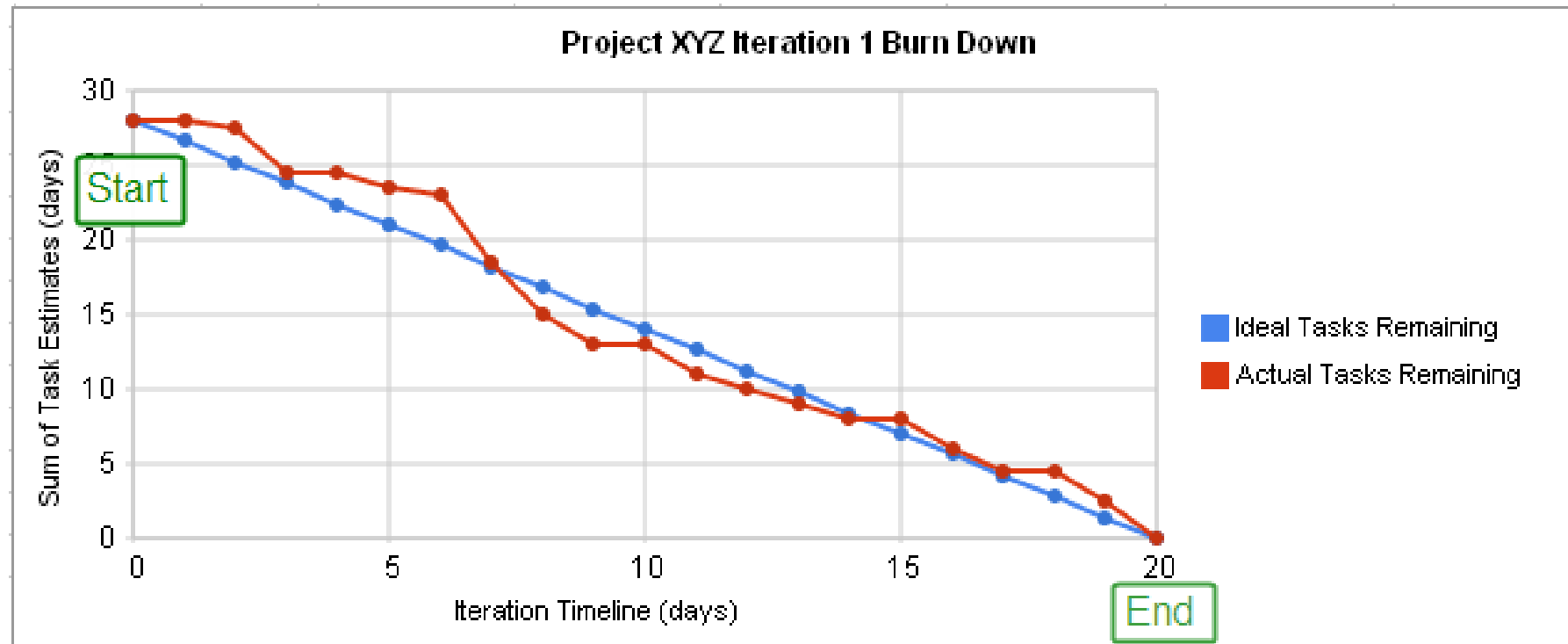
Sprint backlog (real)



Sprint burn-down chart

- Sprint burn-down chart
 - Portrays the work remaining
 - Publically displayed
 - Updated every day
 - “Ideal” is plotted during sprint planning

Sprint burn-down chart (ideal)



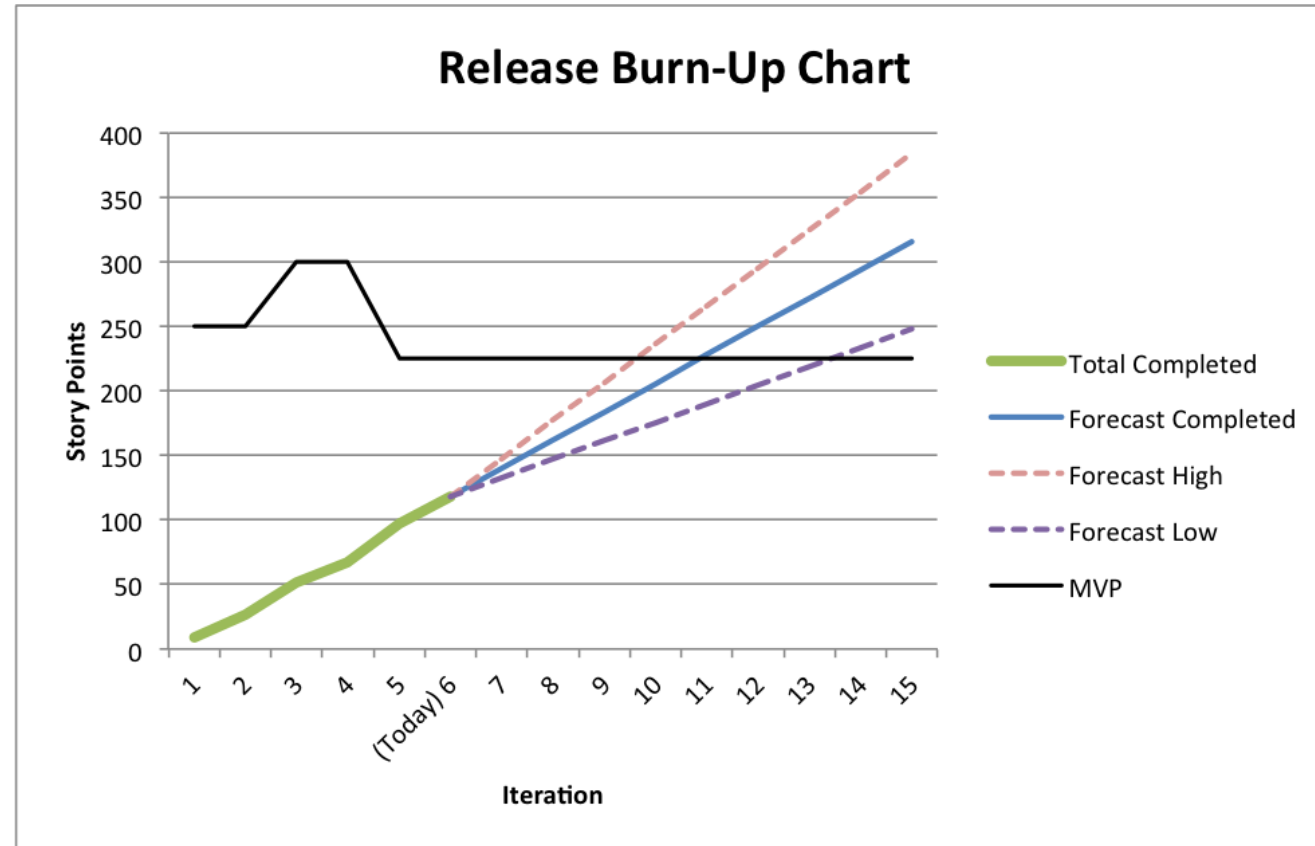
Sprint burn-down chart (real)



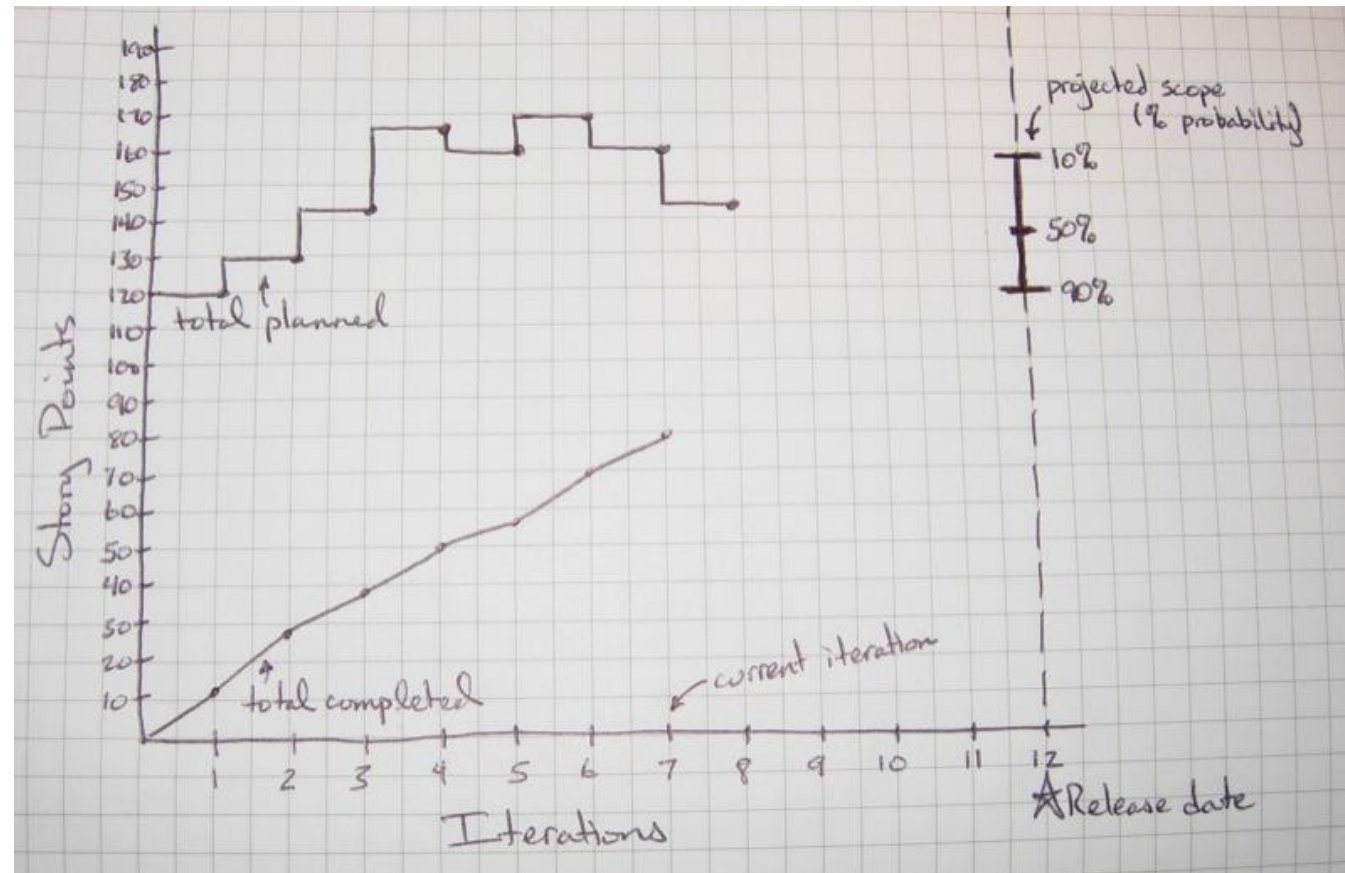
Release burn-up chart

- Release burn-up chart
 - Tracks progress towards a release
 - Updated at the end of each sprint
 - Shows forecasted progress
 - Shows actual progress
 - Shows total work completed/added/removed

Release burn-up chart (ideal)



Release burn-up chart (real-ish)



So, how do we choose?

So .. How do we choose?

- How flexible are your specifications?
- What is your scale and scope?
- Where is your team located?
- How much feedback do you need?
- What is the cost associated with a delay?
- How much change might be requested?

So .. How do we choose?

- The answer ... there is no one right answer
- The software development model that is chosen depends on the project itself, the people running the project, the cost associated with using a particular model, and so on.
- Companies often don't use one development model – they may adopt aspects from several, or use some (but not all) aspects from one
- In fact, companies exist who provide and customise certain development models to suit the needs of their clients (out of the box vs customisation)

COMPX341 - Advanced Software Engineering Design



THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato